

Pathao Courier Merchant API — Integration Specification

Prepared for: MD Belal Hossain (Full-Stack Developer, Node.js/NestJS + Next.js)

Purpose: Hand this document to Claude Code so it can implement a **real, production-ready Pathao Courier integration** on the website/backend. Source: Pathao Merchant Panel → Developer's API (<https://merchant.pathao.com/courier/developer-api>), read in full on 2026-09-09.

1. What this integration does

Pathao Courier is a delivery/logistics provider. The Merchant API lets a website:

1. Authenticate via OAuth2 (password grant) to get an access token.
2. Create a "Store" (a pickup point) in Pathao's system.
3. Look up delivery coverage (city → zone → area) to validate/autocomplete customer addresses.
4. Calculate delivery price before checkout.
5. Create a delivery order (single or bulk) when a customer places an order.
6. Track order status, either by polling or via webhooks Pathao pushes to our server.

Recommended architecture (matches the user's stack): a NestJS `PathaoModule` on the backend that owns all Pathao credentials and calls. The Next.js frontend never talks to Pathao directly — it calls our own backend endpoints, which proxy to Pathao. This is required anyway because `client_secret` must never be exposed to the browser.

2. Environments & Credentials

2.1 Sandbox / Test environment (safe to use immediately, already provisioned by Pathao)

Field	Value
<code>base_url</code>	<code>https://courier-api-sandbox.pathao.com</code>
<code>client_id</code>	<code>7N1aMJQbWm</code>
<code>client_secret</code>	<code>wRcaibZkUdSNz2EI9ZyuXLlNrnAv0TdPU PXMnD39</code>
<code>username</code>	<code>test@pathao.com</code>
<code>password</code>	<code>lovePathao</code>
<code>grant_type</code>	<code>password</code>

These are Pathao's publicly documented sandbox credentials (shared by Pathao with every merchant for testing) — fine to use in `.env.development`, but still keep out of frontend code and out of git if possible.

2.2 Production / Live environment

Field	Value
<code>base_url</code>	<code>https://api-hermes.pathao.com</code>
<code>client_id</code>	Get from Merchant Panel → Developer's API → "Merchant API Credentials" section
<code>client_secret</code>	Get from Merchant Panel → Developer's API → "Merchant API Credentials" section
<code>username / password</code>	Your real Pathao merchant login email/password
<code>grant_type</code>	<code>password</code>

Status as of this writing: the "Merchant API Credentials" panel in the account is currently empty (no live `client_id/client_secret` generated yet). Before going live, log into the merchant panel and generate/copy the live credentials into the production `.env`. Build the integration against sandbox first; only swap `base_url` + credentials for go-live.

2.3 Environment variables to create

.env

PATHAO_BASE_URL=https://courier-api-sandbox.pathao.com

PATHAO_CLIENT_ID=7N1aMJQbWm

PATHAO_CLIENT_SECRET=wRcaibZkUdSNz2EI9ZyuXLINrnAv0TdPUPXMnD39

PATHAO_USERNAME=test@pathao.com

PATHAO_PASSWORD=lovePathao

PATHAO_GRANT_TYPE=password

PATHAO_WEBHOOK_SECRET= # a secret string YOU generate and register in
Pathao's webhook form

PATHAO_STORE_ID= # filled in after "Create a New Store" is called once

3. Authentication (OAuth2)

Pathao uses OAuth2 password grant. All other endpoints require `Authorization: Bearer {access_token}`.

3.1 Issue Access Token

- **POST** `{{base_url}}/aladdin/api/v1/issue-token`
- Headers: `Content-Type: application/json`
- Body:

{

"client_id": "{{client_id}}",

```
"client_secret": "{{client_secret}}",  
  
"grant_type": "password",  
  
"username": "{{your_email}}",  
  
"password": "{{your_password}}"  
  
}
```

- **200 response:**

```
{  
  
"token_type": "Bearer",  
  
"expires_in": 432000,  
  
"access_token": "ISSUED_ACCESS_TOKEN",  
  
"refresh_token": "ISSUED_REFRESH_TOKEN"  
  
}
```

`expires_in` is in seconds (432000s = 5 days).

3.2 Refresh Access Token

- **POST** `{{base_url}}/aladdin/api/v1/issue-token`
- Body:

```
{  
  
"client_id": "{{client_id}}",  
  
"client_secret": "{{client_secret}}",  
  
"grant_type": "refresh_token",  
  
"refresh_token": "ISSUED_REFRESH_TOKEN"  
  
}
```

- Same response shape as 3.1.

3.3 Implementation notes for Claude Code

- Persist `access_token`, `refresh_token`, and a computed `expires_at` (now + `expires_in`) in the database (e.g. a single-row `pathao_tokens` table or a key in Redis) — **do not** re-issue a token on every request.
 - Before each outgoing Pathao API call, check if `expires_at` is within e.g. 5 minutes of now; if so, refresh first (fall back to a full re-issue via username/password if refresh fails).
 - Wrap this in a `PathaoAuthService` with a method `getValidAccessToken(): Promise<string>` that the rest of the module depends on.
 - Add basic retry (1 retry) on `401 Unauthorized` from any Pathao call: refresh the token once and retry the original request.
-

4. Location Data Endpoints (Cities → Zones → Areas)

Used to build cascading dropdowns for pickup/recipient address selection, and required for accurate price calculation.

4.1 Get List of Cities

- **GET** `{{base_url}}/aladdin/api/v1/city-list`
- Headers: `Authorization: Bearer {{access_token}}`
- Response:

```
{ "data": { "data": [ { "city_id": 1, "city_name": "Dhaka" }, { "city_id": 2, "city_name": "Chittagong" } ] } }
```

4.2 Get Zones in a City

- **GET** `{{base_url}}/aladdin/api/v1/cities/{{city_id}}/zone-list`
- Response:

```
{ "data": { "data": [ { "zone_id": 298, "zone_name": "60 feet" } ] } }
```

4.3 Get Areas in a Zone

- **GET** `{{base_url}}/aladdin/api/v1/zones/{{zone_id}}/area-list`
- Response:

```
{ "data": { "data": [ { "area_id": 37, "area_name": "Bonolota", "home_delivery_available": true, "pickup_available": true } ] } }
```

Implementation note: these three are cacheable — city/zone/area lists change rarely. Cache each list in Redis (or in-memory with a TTL of e.g. 24h) keyed by `city_id` / `zone_id` to avoid hammering Pathao on every checkout page load.

5. Store Management

A "Store" in Pathao represents a pickup location. Usually created once (e.g. the business's warehouse/office), and its `store_id` is reused for every order.

5.1 Create a New Store

- **POST** `{{base_url}}/aladdin/api/v1/stores`
- Headers: `Authorization: Bearer {{access_token}}`, `Content-Type: application/json`
- Body:

```
{  
  
  "name": "Demo Store",  
  
  "contact_name": "Test Merchant",  
  
  "contact_number": "017XXXXXXXX",  
  
  "secondary_contact": "015XXXXXXXX",  
  
  "otp_number": "017XXXXXXXX",  
  
  "address": "House 123, Road 4, Sector 10, Uttara, Dhaka-1230, Bangladesh",  
  
  "city_id": 1,  
  
  "zone_id": 298,  
  
  "area_id": 37  
}
```

Field	Type	Required	Notes
name	string	Yes	3–50 chars
contact_name	string	Yes	3–50 chars
contact_number	string	Yes	11 chars
secondary_contact	string	No	11 chars
otp_number	string	No	delivery OTP goes to this number
address	string	Yes	15–120 chars
city_id / zone_id / area_id	integer	Yes	from section 4

- **200 response:** `{ "message": "Store created successfully, Please wait one hour for approval.", "data": { "store_name": "Demo Store" } }`
- **Note:** store approval takes ~1 hour on Pathao's side before it can be used to create orders. This is a one-time setup step, not something to run per-checkout — run it manually once (or from an admin script), save the returned `store_id` (fetch it via 5.2 after approval) into `PATHAO_STORE_ID`.

5.2 Get Merchant Store Info

- **GET** `{{base_url}}/aladdin/api/v1/stores`
- Headers: `Authorization: Bearer {{access_token}}`
- Response includes an array of stores with `store_id`, `store_name`, `store_address`, `is_active`, `city_id`, `zone_id`, `hub_id`, `is_default_store`, `is_default_return_store`.
- Use this once to retrieve the numeric `store_id` after the store above is approved, then hardcode it in env/config — no need to call this on every order.

6. Price Calculation

Call this before order creation to show the customer/admin the delivery fee (e.g. at checkout).

- **POST** `{{base_url}}/aladdin/api/v1/merchant/price-plan`
- Headers: `Authorization: Bearer {{access_token}}`, `Content-Type: application/json`

- Body:

```
{  
  
  "store_id": 12345,  
  
  "item_type": 2,  
  
  "delivery_type": 48,  
  
  "item_weight": 0.5,  
  
  "recipient_city": 1,  
  
  "recipient_zone": 298  
  
}
```

- **200 response:**

```
{  
  
  "data": {  
  
    "price": 80,  
  
    "discount": 0,  
  
    "promo_discount": 0,  
  
    "plan_id": 69,  
  
    "cod_enabled": 1,  
  
    "cod_percentage": 0.01,  
  
    "additional_charge": 0,  
  
    "final_price": 80  
  
  }  
  
}
```

- `final_price` is what should be shown to the customer as the delivery charge.
-

7. Order Creation

7.1 Create a Single Order

- **POST** `{{base_url}}/aladdin/api/v1/orders`
- Headers: `Authorization: Bearer {{access_token}}`, `Content-Type: application/json`
- Body:

```
{  
  
  "store_id": 12345,  
  
  "merchant_order_id": "ORDER-1001",  
  
  "recipient_name": "Demo Recipient",  
  
  "recipient_phone": "017XXXXXXXXX",  
  
  "recipient_address": "House 123, Road 4, Sector 10, Uttara, Dhaka-1230, Bangladesh",  
  
  "delivery_type": 48,  
  
  "item_type": 2,  
  
  "special_instruction": "Need to Delivery before 5 PM",  
  
  "item_quantity": 1,  
  
  "item_weight": "0.5",  
  
  "item_description": "this is a Cloth item, price- 3000",  
  
  "amount_to_collect": 900  
}
```

Field	Type	Required	Notes
store_id	integer	Yes	from section 5
merchant_order_id	string	No	our own order id/reference — always send this so we can reconcile
recipient_name	string	Yes	3–100 chars
recipient_phone	string	Yes	11 chars
recipient_secondary_phone	string	No	11 chars
recipient_address	string	Yes	10–220 chars
recipient_city / recipient_zone / recipient_area	integer	No	omit entirely (don't send null) to let Pathao auto-detect from address; send explicitly if we already collected them via section 4 dropdowns for accuracy
delivery_type	integer	Yes	48 = Normal Delivery, 12 = On Demand Delivery
item_type	integer	Yes	1 = Document, 2 = Parcel
special_instruction	string	No	
item_quantity	integer	Yes	
item_weight	float	Yes	0.5–10 kg
item_description	string	No	
amount_to_collect	integer	Yes	COD amount to collect from recipient; 0 if not COD

- **200 response:**

```
{
  "message": "Order Created Successfully",
  "data": {
    "consignment_id": "DL121224VS8TTJ",
    "merchant_order_id": "ORDER-1001",
    "order_status": "Pending",
    "delivery_fee": 80
  }
}
```

- **consignment_id** is Pathao's tracking ID — store it on our own order record.

7.2 Create a Bulk Order

- **POST** `{{base_url}}/aladdin/api/v1/orders/bulk`
- Headers: `Authorization: Bearer {{access_token}}`, `Content-Type: application/json; charset=UTF-8`
- Body: `{ "orders": [ { ...same shape as 7.1... }, { ... } ] }`
- **202 response** (accepted, async processing): `{ "message": "...", "code": 202, "data": true }` — note this does **not** return `consignment_ids` directly; poll 7.3 or rely on the `order.created` webhook per order to get consignment IDs.

7.3 Get Order Short Info

- **GET** `{{base_url}}/aladdin/api/v1/orders/{{consignment_id}}/info`
 - Headers: `Authorization: Bearer {{access_token}}`
 - Response: `{ "data": { "consignment_id", "merchant_order_id", "order_status", "order_status_slug", "updated_at", "invoice_id" } }`
 - Use for on-demand status refresh (e.g. an admin "refresh status" button); prefer webhooks (section 8) for real-time updates instead of polling.
-

8. Webhook Integration (real-time order status updates)

This is the recommended way to keep order status in sync, instead of polling.

8.1 Setup (one-time, done in the Pathao merchant panel by a human, not code)

In Merchant Panel → Developer's API → Webhook Integration, register:

- **Callback URL:** a public HTTPS endpoint on our backend, e.g. `POST https://our-domain.com/api/webhooks/pathao`
- **Secret:** a string we generate ourselves (save it as `PATHAO_WEBHOOK_SECRET`)
- **Events:** select the events we care about (see catalog below) or "Select All"

8.2 Endpoint verification handshake

When we register the callback URL, Pathao immediately sends a verification POST:

```
{ "event": "webhook_integration" }
```

Our endpoint **must**, for this request:

- Respond within 10 seconds
- Return HTTP status **202**
- Return a response header `X-Pathao-Merchant-Webhook-Integration-Secret` whose value is exactly `f3992ecc-59da-4cbe-a049-a13da2018d51` (this literal value came from Pathao's own docs page — it's Pathao's own verification token to prove we handled their handshake, not a per-merchant secret)

General requirements for the callback URL at all times:

- Publicly reachable, resolves within 3 redirects, valid SSL if HTTPS
- Responds within 10 seconds
- Returns status 202 for actual events

8.3 Headers Pathao sends on real events

Header	Value
<code>X-PATHAO-Signature</code>	The secret we provided during setup (<code>PATHAO_WEBHOOK_SECRET</code>) — verify this matches before trusting the payload

Header	Value
Content-Type	application/json

8.4 Payload shape

Common fields on every event, plus event-specific extra fields:

```
{  
  
  "consignment_id": "DL121224VS8TTJ",  
  
  "merchant_order_id": "TS-123",  
  
  "updated_at": "2024-12-27 23:49:43",  
  
  "timestamp": "2024-12-27T17:49:43+00:00",  
  
  "store_id": 130820,  
  
  "event": "order.created",  
  
  "delivery_fee": 83.46  
}
```

Example for `order.delivered` (extra field `collected_amount` instead of `delivery_fee`):

```
{  
  
  "consignment_id": "DL121224VS8TTJ",  
  
  "merchant_order_id": "TS-123",  
  
  "updated_at": "2024-12-27 23:53:23",  
  
  "timestamp": "2024-12-27T17:49:43+00:00",  
  
  "store_id": 130820,  
  
  "event": "order.delivered",  
  
  "collected_amount": 60  
}
```

```
}
```

Treat **event-specific extra fields as optional/variable** — parse `consignment_id`, `merchant_order_id`, `event`, `updated_at/timestamp`, `store_id` as the guaranteed base shape, and read any additional fields defensively (they differ per event type).

8.5 Full event catalog (selectable in the merchant panel)

Order Created, Order Updated, Pickup Requested, Assigned For Pickup, Pickup, Pickup Failed, Pickup Cancelled, At the Sorting Hub, In Transit, Received at Last Mile Hub, Assigned for Delivery, Delivered, Partial Delivery, Return, Delivery Failed, On Hold, Payment Invoice, Paid Return, Exchange, Store Created, Store Updated, Return Id Created, Return In Transit, Returned To Merchant.

8.6 Implementation notes for Claude Code

- Build one NestJS controller route, e.g. `POST /webhooks/pathao`, **excluded from any global auth guard** (Pathao can't send our JWT) but protected by verifying `X-PATHAO-Signature === PATHAO_WEBHOOK_SECRET`.
- Handle the `{"event": "webhook_integration"}` handshake case explicitly first (see 8.2) — return 202 + the required header immediately, don't try to process it as a real order event.
- For real events, look up our local order by `merchant_order_id` (preferred) or `consignment_id`, update its status/timeline, and respond 202 quickly — do any slow work (emails, notifications) asynchronously (e.g. a queue) so we don't risk timing out Pathao's 10-second window.
- Make the handler idempotent (Pathao may retry deliveries) — e.g. upsert an `order_status_events` row keyed by `(consignment_id, event, updated_at)` before acting on it.
- Log unrecognized `event` values instead of throwing, since Pathao may add new event types over time.

9. Suggested NestJS module layout

`src/`

`pathao/`

`pathao.module.ts`

```
pathao-config.ts      # reads env vars, validated with e.g. zod/class-validator

pathao-auth.service.ts  # token issue/refresh/cache (section 3)

pathao-location.service.ts # cities/zones/areas + Redis caching (section 4)

pathao-store.service.ts  # create/get store (section 5)

pathao-order.service.ts  # price-plan, create order, bulk, get info (sections 6–7)

pathao-webhook.controller.ts # webhook receiver (section 8)

pathao.http.ts          # thin axios/fetch wrapper: base_url + bearer header + 401 retry-once

dto/

  create-order.dto.ts

  price-plan.dto.ts

  webhook-event.dto.ts
```

Suggested TypeScript types Claude Code can start from:

```
export type PathaoDeliveryType = 48 | 12; // 48 Normal, 12 On Demand

export type PathaoItemType = 1 | 2;    // 1 Document, 2 Parcel

export interface CreatePathaoOrderInput {

  store_id: number;

  merchant_order_id?: string;

  recipient_name: string;

  recipient_phone: string;

  recipient_secondary_phone?: string;

  recipient_address: string;
```

```
recipient_city?: number;

recipient_zone?: number;

recipient_area?: number;

delivery_type: PathaoDeliveryType;

item_type: PathaoItemType;

special_instruction?: string;

item_quantity: number;

item_weight: string; // e.g. "0.5"

item_description?: string;

amount_to_collect: number;

}

export interface PathaoWebhookBasePayload {

  consignment_id: string;

  merchant_order_id?: string;

  updated_at: string;

  timestamp: string;

  store_id: number;

  event: string;

  [extra: string]: unknown; // event-specific fields vary

}
```

10. End-to-end flow to implement

1. **App startup / cron:** `PathaoAuthService` ensures a valid cached access token (issue once, refresh before expiry).
 2. **One-time admin setup:** call Create Store (5.1) once, wait ~1h for approval, fetch `store_id` via 5.2, save to config.
 3. **Checkout UI:** cascading City → Zone → Area selects (4.1–4.3, cached), then call Price Calculation (section 6) to show delivery fee before the customer confirms.
 4. **Order placement:** after our own order is saved, call Create Order (7.1) with our `merchant_order_id`; store the returned `consignment_id` and `delivery_fee` on our order record.
 5. **Status sync:** rely on the webhook (section 8) to update order status in real time; use Get Order Short Info (7.3) only as an on-demand fallback/refresh.
 6. **Admin/support tooling:** optionally surface Bulk Order (7.2) for batch fulfillment days.
-

11. Security checklist

- `client_secret`, `PATHAO_WEBHOOK_SECRET`, and tokens live only on the backend — never sent to the Next.js client bundle.
 - Recompute `amount_to_collect` and delivery fee server-side from our own order/cart data — never trust a client-submitted amount.
 - Verify `X-PATHAO-Signature` on every webhook call before trusting the payload.
 - Rate-limit / cache the location endpoints (section 4) — they don't need to be called on every keystroke.
 - Log Pathao API errors (non-200 responses) with enough context (`merchant_order_id`) to manually retry/investigate failed order creations.
-

12. Reference: other panel resources noted but out of scope for API integration

- WordPress Plugin and Shopify Plugin exist as no-code alternatives (not needed here since this is a custom Node.js/Next.js build).
- Pricing Plan / Coverage Area / Promo pages in the merchant panel are informational/admin pages, not part of the developer API surface.